

JavaScript-06

Postman

30 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg

JS

🌐 Status: 200 OK Time: 116 ms

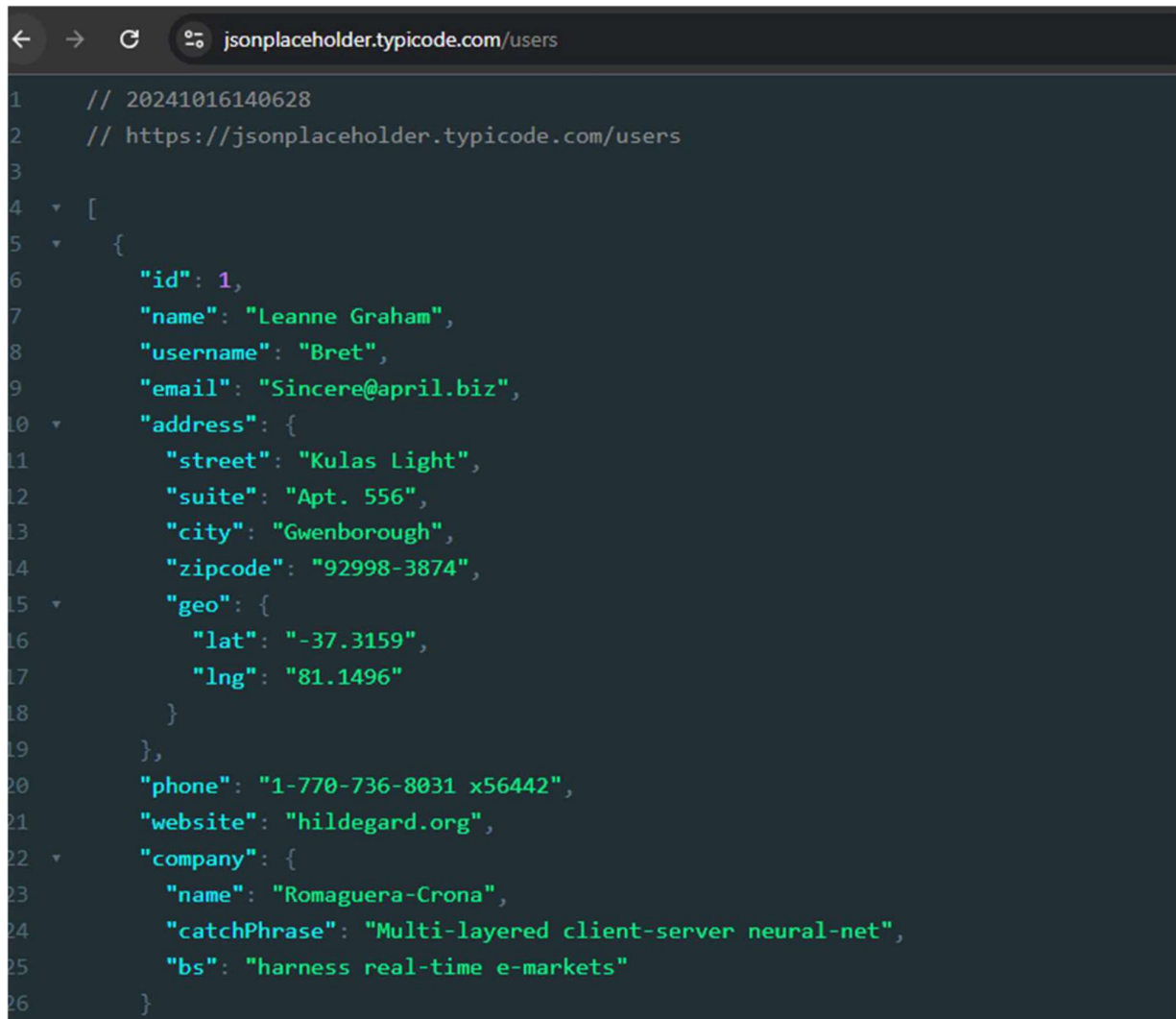


För en kanin

Postman

Fakta om Postman

När man utvecklar webbapplikationer och vill testa anrop mot REST-API:er är Postman ett mycket bra verktyg. Man vill gärna se vad man får tillbaka som retur från ett REST API och också testa hur det går att skicka till detsamma.



```
1 // 20241016140628
2 // https://jsonplaceholder.typicode.com/users
3
4 [
5   {
6     "id": 1,
7     "name": "Leanne Graham",
8     "username": "Bret",
9     "email": "Sincere@april.biz",
10    "address": {
11      "street": "Kulas Light",
12      "suite": "Apt. 556",
13      "city": "Gwenborough",
14      "zipcode": "92998-3874",
15      "geo": {
16        "lat": "-37.3159",
17        "lng": "81.1496"
18      }
19    },
20    "phone": "1-770-736-8031 x56442",
21    "website": "hildegard.org",
22    "company": {
23      "name": "Romaguera-Crona",
24      "catchPhrase": "Multi-layered client-server neural-net",
25      "bs": "harness real-time e-markets"
26    }
27  }
28 ]
```

Bild 1 En lista med JSON-objekt från jsonplaceholder.

Om man endast behöver köra HTTP request metoden GET så klarar man sig faktiskt utmärkt med bara en vanlig webbläsare som Chrome eller Edge. Du skriver helt enkelt in adressen till ditt REST API och trycker på enter. Då ser det ut som på bild 1 om allting är korrekt. Här är det en GET request till <https://jsonplaceholder.typicode.com/users>. Det går däremot inte att göra POST, PUT, PATCH eller DELETE i en webbläsare. För det behöver man Postman för att hantera det.

Ladda ner

Det går att köra Postman både som webbapplikation och desktop-applikation. För den här manualen används desktop-applikationen. Allting borde se exakt lika ut men det är inte garanterat att det är så. Börja med att gå till länken <https://postman.com/downloads/> och ladda ner den versionen som hör till ditt operativsystem. När nedladdningen är färdig så kan du starta applikationen och skapa ett konto. Det är några funktioner som inte går att komma åt om man inte är inloggad.

Det kostar ingenting att använda Postman om man kör det som studerande och inte har det som ett företag. När applikationen har startat kan du gå in på inställningar uppe i högra hörnet, enligt bild 2. Där klickar du på Settings och sen kan du välja ett tema, mörkt eller ljust, eller välja att köra på det som är inställt på din dator.

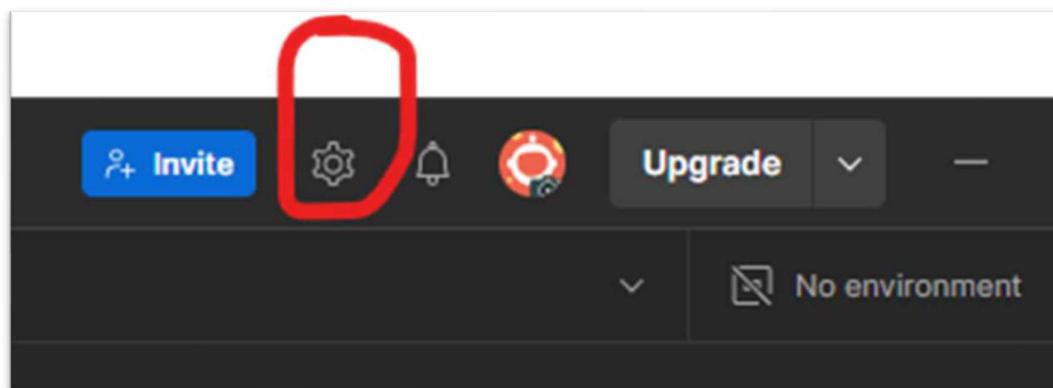


Bild 2 Inställningar uppe i högra hörnet.

Till vänster finns det några ikoner. Klicka på den som det står Collections på enligt bild 3.

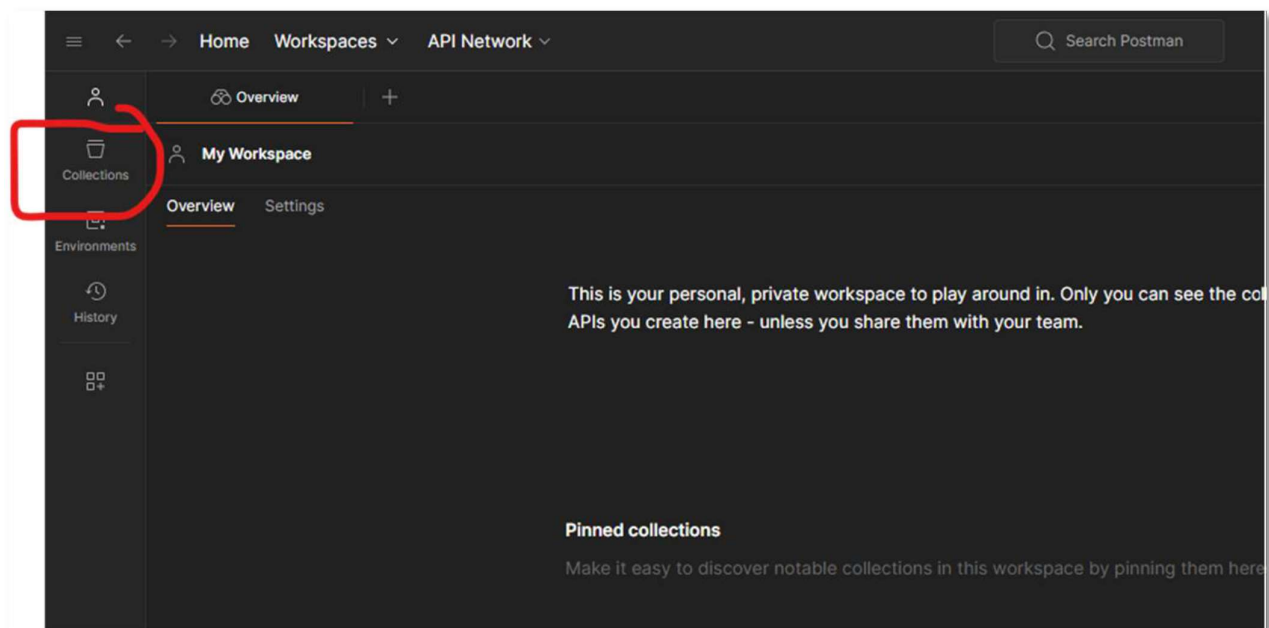


Bild 3 Ikonen Collections till vänster i applikationen.

Nu kan vi skapa en Collection för att samla alla typer av anrop som gör mot en och samma REST API. Klicka på plus-ikonen överst i Collections och välj en Blank collection. Nu bör det dyka upp en New Collection i listan under plus-ikonen. Om man för musen ovanför den så dyker det upp en 3-prickar-ikon som man kan klicka på. I den listan som dyker upp så väljer vi Rename och ger vår nya Collection ett bra namn. Till exempel JSONPlaceholder eftersom vi kommer att träna mot den REST API:n. Nu bör det se ut som på bild 4.

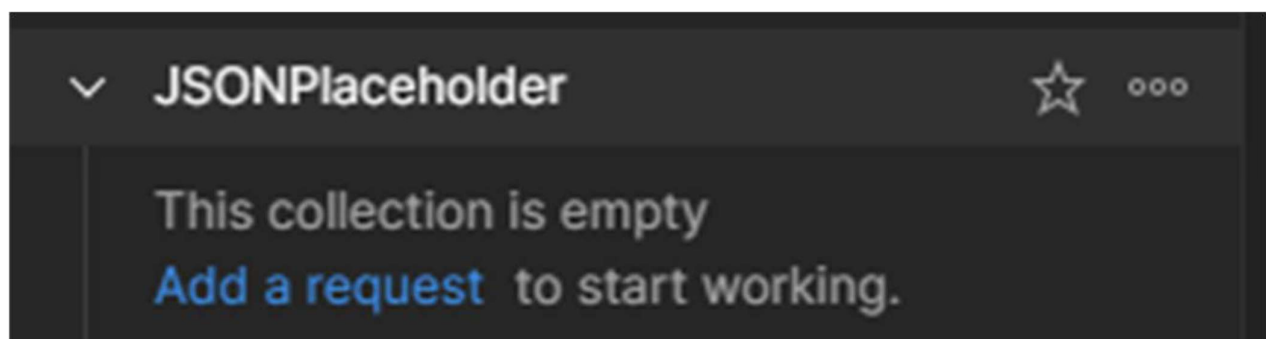


Bild 4 Vår nya omdöpta Collection.

En request

Vi försöker göra vår första request genom att antingen klicka på den blåa länken enligt bild 4. Man kan också föra musen över vår Collection, JSONPlaceholder tills man ser ikonen med 3 prickar och klicka på Add request (ligger som sjätte val uppifrån). Nu skapas det en ny flik på den högra delen av applikationen, se bild 5.

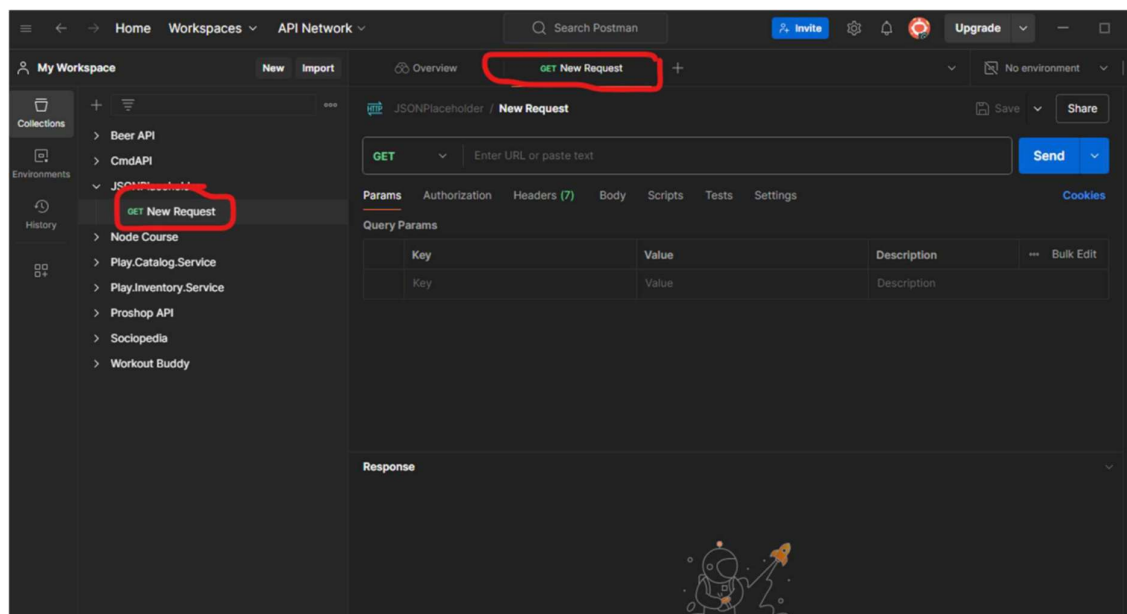


Bild 5 Den nya fliken med vår request.

Man kan ändra namn på den här requesten genom att föra musen över New Request till höger. När de 3 prickarna dyker upp klickar man på den och väljer Rename som finns ungefär i mitten. Här kan man skriva in vad det är requesten gör, till exempel Get all users eller Hämta alla användare. Nu flyttar vi fokus till det stora området till höger, enligt bild 6.

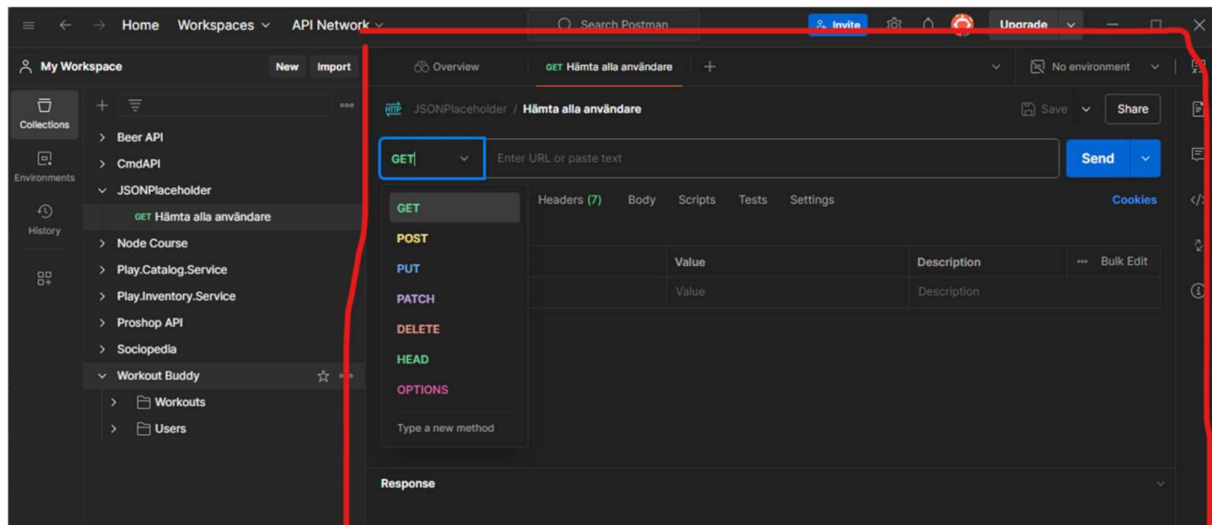


Bild 6 Stora arbetsområdet.

Se till att din nya request-flik är markerad. Under fliken och namnet på din request finns först en dropdown-meny där man kan ställa vilken typ av request man vill göra. Sedan finns det ett långt input-fält där man skriver in adressen till det REST-API man vill anropa. Slutligen på samma rad finns en blå knapp där man skickar iväg sin request när man har fyllt i allting. För den request vi ska göra först borde det se ut som på bild 7.

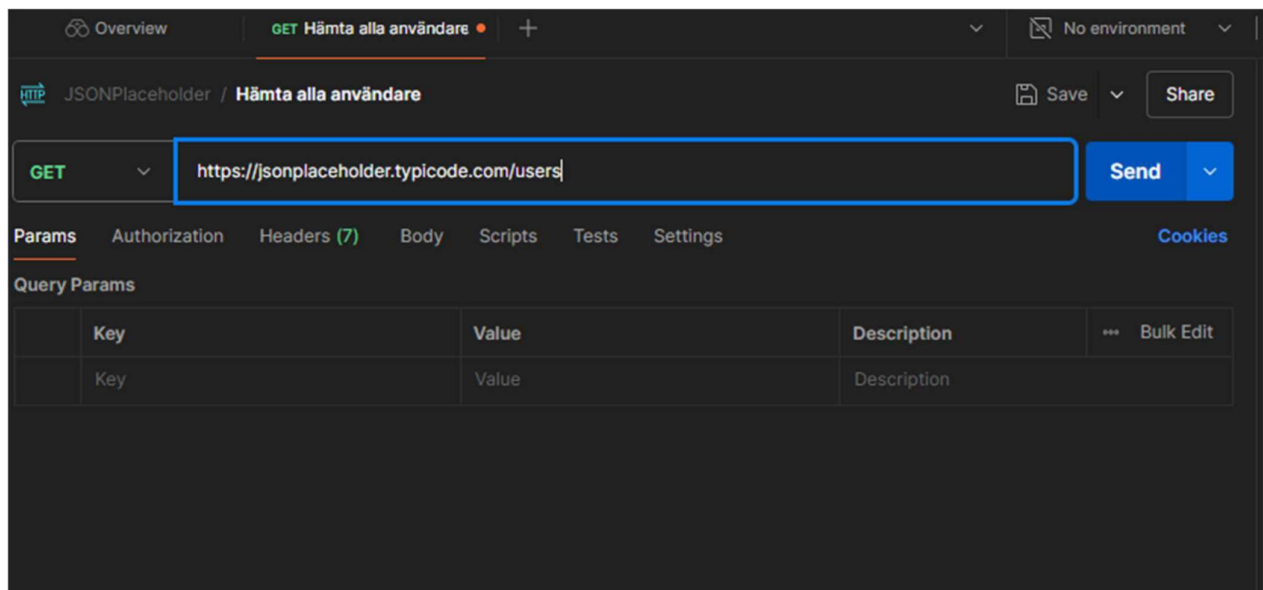


Bild 7 Typ av request och url för requesten.

Innan vi trycker iväg vår request kan vi trycka på ctrl + S och se till att vår request är sparad. När vi nu trycker på knappen Send så står det Sending request på den nedre delen av det

stora området. Man kan dra upp det nedre området och göra det lite större. Men det är här vi ser vilket svar vi får från REST API:et, se bild 8.

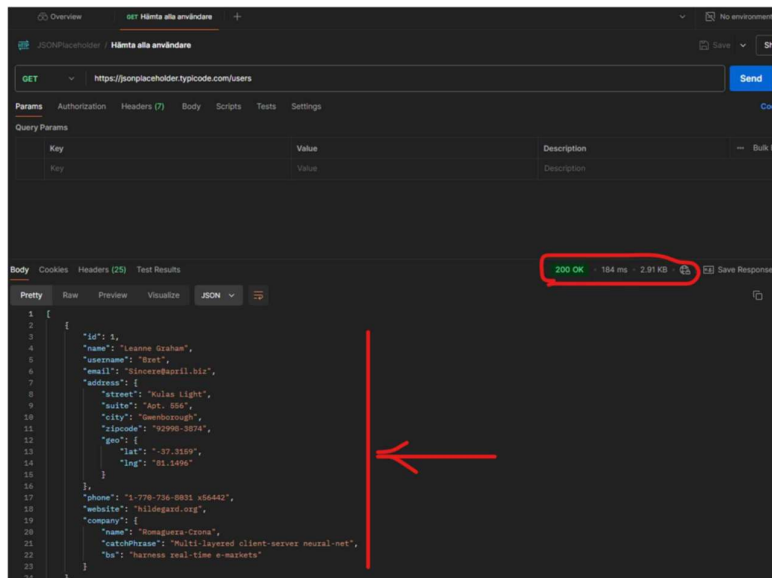


Bild 8 Svarsdelen i Postman

Dels är det intressant att titta uppe till höger för att se att man har fått ett grönt 200 OK. Det innebär att vi har gjort rätt och REST API:et också mår bra. Om man har en annan kod än 200 och det är rött där, då har man gjort fel eller så är REST API:et inte fungerande. Det kan vara olika typer av server-fel eller fel i anropet. Det kan också innebära att man har glömt att infoga en API-nyckel som ibland krävs. Det andra intressanta att titta på är ju själva svaret, om man nu har fått 200 OK. Det finns lite mer till vänster och det är i det här fallet en lista med 10 objekt som har kommit tillbaka från servern.

REST API-dokumentation

När man skriver ett REST API är det viktigt att man skriver bra dokumentation för användarna. Det kan vara svårt att veta om alla sätt det går att kommunicera med servern om man inte har varit med och utvecklat det själv. För det REST API som vi jobbar med finns det dokumentation på länkarna <https://jsonplaceholder.typicode.com/guide/>. Om man till exempel vill hämta alla poster från kan man anropa url:en

<https://jsonplaceholder.typicode.com/posts> med en GET request. Det brukar man kalla för en endpoint. Till den endpointen kan man också skicka POST request. Om man vill hämta information om en specifik post kan man anropa url:en

<https://jsonplaceholder.typicode.com/posts/1> där den sista 1:an i adressen anger vilket id-

nummer man är intresserad av. Det är alltså inte samma endpoint som den förra, även fast de är snarlika. Men till den ena skickar man alltså med en extra svans, ett id som för denna REST API är i form av ett tal. Ibland kan ett id vara i form av en sträng också. Det är därför det är viktigt att gå in och läsa dokumentationen för REST API:et så att man vet hur servern förväntar sig att bli anropad.

Swagger

Det finns program för att skriva dokumentation för REST API:er, bland annat ett som heter Swagger. Det är mycket möjligt att man stöter på sådana ibland. Det finns ett exempel på ett sådant på nätet <https://petstore.swagger.io/> som alltså är ett så kallat låtsas-API. Fördelen med en dokumentation skriven i Swagger är att man kan testa runt med olika anrop utan att behöva använda Postman. Om vi provar dokumentationen på petstore.swagger.io så ser man att det är uppdelat på kategorierna pet, store och user, enligt bild 9.

Swagger. Supported by SMARTBEAR

https://petstore.swagger.io/v2/swagger.json Explore

Swagger Petstore 1.0.7 OAS 2.0

[Base URL: petstore.swagger.io/v2]
<https://petstore.swagger.io/v2/swagger.json>

This is a sample server Petstore server. You can find out more about Swagger at <http://swagger.io> or on [irc.freenode.net, #swagger](irc://freenode.net/swagger). For this sample, you can use the api key `special-key` to test the authorization filters.

[Terms of service](#)
[Contact the developer](#)
[Apache 2.0](#)
[Find out more about Swagger](#)

Schemes
HTTPS

Authorize

- pet** Everything about your Pets [Find out more](#)
- store** Access to Petstore orders
- user** Operations about user [Find out more about our store](#)

Bild 9 Kategorierna pet, store och user.

Säg att vi vill ta reda på hur det här REST API:et fungerar. Vi kan börja med att kolla på kategorin pet, Det finns fyra olika endpoints, men några av dem har olika requestmetoder. För endpointen `/pet/{petId}` finns det antingen GET, POST eller DELETE, och för endpointen `/pet` finns det antingen POST eller PUT. Vi ska försöka hitta lite information om ett husdjur, så då vill vi såklart använda GET med endpointen `/pet/{petId}`, enligt bild 10.

pet Everything about your Pets		Find out more ^
POST	/pet/{petId}/uploadImage uploads an image	🔒 ▼
POST	/pet Add a new pet to the store	🔒 ▼
PUT	/pet Update an existing pet	🔒 ▼
GET	/pet/findByStatus Finds Pets by status	🔒 ▼
GET	/pet/findByTags Finds Pets by tags	🔒 ▼
GET	/pet/{petId} Find pet by ID	🔒 ▼
POST	/pet/{petId} Updates a pet in the store with form data	🔒 ▼
DELETE	/pet/{petId} Deletes a pet	🔒 ▼

Bild 10 Endpoint /pet/{petId}

Det ser lite konstigt ut med petId skrivet innanför klammerparenteser. Tanken är att man ska byta ut det mot ett nummer. Om man klickar på raden som är markerad i bild 10 så finns det några delar som är bra att reda ut, se bild 11.

GET /pet/{petId} Find pet by ID

Returns a single pet

Try it out

Parameters

Name	Description
petId * required integer(\$int64) (path)	ID of pet to return

3532

Responses

Response content type: application/json

Curl

```
curl -X 'GET' \
'https://petstore.swagger.io/v2/pet/3532' \
-H 'accept: application/json'
```

Request URL

```
https://petstore.swagger.io/v2/pet/3532
```

Server response

Code	Details
200	Response body <pre>{ "id": 3532, "category": { "id": 8031, "name": "Border Collie" }, "name": "Shadow", "photoUrls": ["temporibus_necessitatibus\\illum.wav"], "tags": [{ "id": 0, "name": "Jake" }] }</pre>

Bild 11 Förklaring av endpoint-information.

Överst för all information finns det en kort beskrivning av vad den här endpointen skickar tillbaka. Den returnerar alltså ett endaste husdjur. Till höger finns det en inringad knapp som heter Try it out. Den kan man klicka på för att möjliggöra testläge av endpointen. När man klickar på den dyker det upp två knappar till. En som heter Execute och en som heter Clear. Strax ovanför knappen Execute finns det en förklaring av den parameter som vi behöver

skicka med, alltså {petId}. Det står att den ska vara med i form av ett heltal (integer). Så förklaringen till det något kryptiska /pet/{petId} är att man ska skriva /pet/3532 om man vill veta mer om husdjuret med petId 3532. Vill man veta lite mer om husdjur med petId 24 skriver man /pet/24. Nu finns inte alla petId-nummer i REST-API:et. Men vi vill kolla upp information om något husdjur. Skriv in siffran 1 (eller något annat lågt nummer) i rutan för **petId**. Klicka sedan på den stora blåa knappen **Execute**.

Swagger kommer nu att skicka ett riktigt API-anrop i bakgrunden. Om du scrollar ner en liten bit till sektionen *Server response* ser du resultatet, enligt bild 12.

Code	Details
200	Response body (Här ser du husdjurets namn, kategori och status i JSON-format)

Bild 12 Svaret från Swagger-servern som visar data i JSON-format.

Om du istället skriver in ett jättehögt, slumpmässigt nummer (som förmodligen inte finns i databasen) och trycker på Execute, kommer du att få se en röd eller gul ruta med **Code 404 (Not Found)**. Swagger är alltså ett perfekt verktyg för att snabbt se hur ett API reagerar på både rätt och felaktig data innan du börjar skriva din egen JavaScript-kod!

Skapa ett husdjur med POST i Postman

Nu när vi har sett hur ett husdjur ser ut i Swagger, ska vi använda **Postman** för att skicka data och registrera ett helt nytt husdjur i databasen med hjälp av metoden POST.

Följ dessa steg noggrant för att göra ditt första POST-anrop:

Steg 1: Skapa en ny Request i Postman

1. Öppna Postman och gå till din Collection.
2. Högerklicka på din Collection och välj **Add request**.
3. Ge din nya request namnet Skapa husdjur och ändra metoden i rullistan till vänster från GET till **POST**.
4. I URL-fältet skriver du in endpointen för att skapa husdjur: <https://swagger.io>

Steg 2: Ställ in din "Body" till JSON

Eftersom vi ska skicka data måste vi berätta för Postman vilket format vi vill skriva i.

1. Klicka på fliken **Body** (som ligger precis under det långa URL-fältet).
2. Välj alternativet **raw** i radioknapparna.
3. Längst till höger ändrar du texttypen i rullistan från *Text* till **JSON**, enligt bild 13.

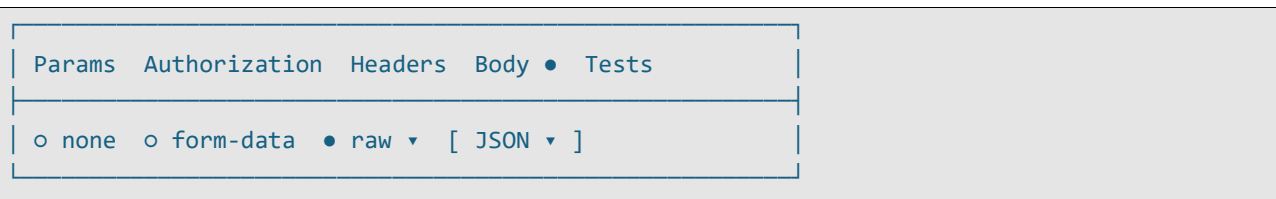


Bild 13 Inställningar under fliken Body för att skicka data som JSON.

Steg 3: Skriv in ditt husdjur

I det stora textfältet som dök upp ska du nu klistra in data för det husdjur du vill skapa. Kopiera texten nedan och klistra in den i Postman. Du får självklart ändra namnet "Buster" till vad du vill.

```
{
  "id": 987654321,
  "category": {
    "id": 1,
    "name": "Hundar"
  },
  "name": "Buster",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "Snäll"
    }
  ],
  "status": "available"
}
```

Bild 14 Objektet som skickas med i POST-anropet.

Steg 4: Skicka och kontrollera svaret

När du har klistrat in koden trycker du på **Send**. Titta nu i det nedre högra hörnet av svarsfönstret enligt bild 10.

Status: 200 OK Time: 145 ms

Bild 15 Statuskoden uppe till höger i Postmans svarsfönster.

Om du får **Status: 200 OK** (eller 201 Created på vissa API:er) betyder det att din data har skickats iväg, passerat serverns dörrvakter och sparats i databasen! I svarsrutan till vänster ser du det färdiga objektet som servern skickade tillbaka till dig som en bekräftelse.

Utmaning för studenten (Frivillig)

För att bevisa att ditt husdjur faktiskt lever i databasen:

1. Skapa en ny **GET**-request i Postman.
 2. Anropa adressen <https://swagger.io> (byt ut numret i slutet mot det ID du valde i steg 3).
 3. Tryck på Send. Om du får tillbaka ditt husdjur har du lyckats konfigurera dina första API-flöden i Postman på en professionell nivå!
-